

Java Multithreading and Networking Notes

These notes explain core concepts of multithreading and networking in Java in a simple and structured manner with examples and code.

1. Thread

A thread is a unit of execution within a program. Every Java program starts with a main thread. Multithreading allows multiple parts of a program to run at the same time. This helps in improving performance and making applications responsive.

2. Need of Multithreading

Multithreading is used when a program needs to perform multiple tasks simultaneously. Examples:
- A music player playing songs while downloading another file
- A web server handling multiple users at the same time

3. Creating Threads

Using Thread class:

```
class MyThread extends Thread {
    public void run() {
        System.out.println("Thread running...");
    }

    public static void main(String[] args) {
        MyThread t = new MyThread();
        t.start();
    }
}
```

Using Runnable interface:

```
class MyTask implements Runnable {
    public void run() {
        System.out.println("Thread using Runnable");
    }

    public static void main(String[] args) {
        Thread t = new Thread(new MyTask());
        t.start();
    }
}
```

Runnable is preferred because it allows better flexibility and avoids inheritance limitations.

4. Thread Life Cycle

A thread goes through different states during execution: New → Runnable → Running → Waiting/Blocked → Terminated - New: Thread object created - Runnable: Ready to run - Running: Currently executing - Waiting: Waiting for resource or signal - Terminated: Execution finished

5. Thread Priority

Each thread has a priority from 1 to 10. Higher priority increases chances of execution, but it is not guaranteed as it depends on the operating system.

```
Thread t = new Thread();
t.setPriority(Thread.MAX_PRIORITY);
```

6. Synchronization

When multiple threads access shared data, it can lead to incorrect results. Synchronization ensures only one thread accesses the critical section at a time.

```
class Bank {
    int balance = 1000;

    synchronized void withdraw(int amount) {
        if(balance >= amount) {
            balance -= amount;
            System.out.println("Remaining: " + balance);
        }
    }
}
```

Example: If two users try to withdraw money at the same time, synchronization prevents incorrect balance updates.

7. Inter-Thread Communication

Threads can communicate using: - wait() - notify() - notifyAll() These methods help coordinate execution between threads.

8. Producer-Consumer Problem

One thread produces data and another consumes it. Proper coordination is required to avoid data inconsistency.

```
class Shared {
    int data;
    boolean available = false;

    synchronized void produce(int value) throws Exception {
        while(available) wait();
        data = value;
        available = true;
        notify();
    }

    synchronized void consume() throws Exception {
        while(!available) wait();
        available = false;
    }
}
```

```

        }
        notify();
    }
}

```

9. Networking Basics

Networking allows communication between different systems. Client: sends request Server: processes request and sends response Example: Browser (client) → Web server → Response

10. TCP Protocol

TCP is a reliable protocol that ensures: - Data is delivered correctly - Data is delivered in order - No data loss occurs

11. Socket Programming

Socket is an endpoint used for communication between client and server.

```

// Server
import java.net.*;
class Server {
    public static void main(String[] args) throws Exception {
        ServerSocket ss = new ServerSocket(5000);
        Socket s = ss.accept();
        System.out.println("Client connected");
    }
}

// Client
import java.net.*;
class Client {
    public static void main(String[] args) throws Exception {
        Socket s = new Socket("localhost", 5000);
        System.out.println("Connected to server");
    }
}

```

12. Client-Server Communication Flow

Client → Request → Server → Processing → Response → Client

13. Real World Usage

Multithreading: - Mobile apps - Web servers - Games Networking: - Websites - APIs - Chat applications